Tutorial IV.I - Introduction to Mathematical Optimization with JuMP and HiGHS

Applied Optimization with Julia



Introduction

Welcome to this beginner-friendly tutorial on mathematical optimization using JuMP and the HiGHS solver in Julia! Don't worry if these terms sound unfamiliar – we'll explain everything step by step.

In this tutorial, you'll learn how to:

1. Set up a simple optimization problem
2. Define variables and constraints
3. Create an objective function
4. Solve the problem and interpret the results

We'll use a real-world example to make these concepts more relatable. Imagine you're managing a small factory that produces two types of products. Your goal is to maximize profit while working within certain limitations. This is exactly the kind of problem that mathematical optimization can solve!

What is JuMP?

JuMP (Julia for Mathematical Programming) is a powerful tool that helps us describe optimization problems in a way that computers can understand and solve. Think of it as a translator between your business problem and the mathematical solver.

What is HiGHS?

HiGHS is an open-source solver that can find solutions to the optimization problems we describe using JuMP. It's like a very smart calculator that can handle complex problems quickly and efficiently.

Our Example Problem

Let's break down our factory management problem:

- You produce two products: Product A and Product B
- Each product gives you a different profit:
  - Product A: 100 profit per unit
  - Product B: 150 profit per unit
- You have two departments: Cutting and Finishing
- Each product requires different amounts of time in each department:

  - Product A: 2 hours in Cutting, 4 hours in Finishing
  - Product B: 4 hours in Cutting, 3 hours in Finishing
- You have limited time available in each department:
  - Cutting: 40 hours total
  - Finishing: 60 hours total

Your goal is to decide how many of each product to make to maximize your total profit, while not exceeding the available time in each department.

In the lecture's game, the two products are skateboards (product A) and surfboards (product B): the same workshop, with the same numbers.

Setting Up

First, we need to install and load the necessary packages. If you haven't already installed JuMP and HiGHS, you can do so by running `import Pkg` and then `Pkg.add(["JuMP","HiGHS"])`.

Now, let's load these packages:

```
using JuMP, HiGHS
```

Great! We're now ready to start building our optimization model.

Section 1 - Defining an Optimization Model

The first step in solving an optimization problem is to create a model. This model will contain all the information about our problem: the variables, constraints, and objective.

In JuMP, we create a model like this:

```
model = Model(HiGHS.Optimizer)
println("Optimization model created successfully!")
```

```
Optimization model created successfully!
```

Let's break this down: - `Model()` creates a new optimization model - `HiGHS.Optimizer` tells JuMP to use the HiGHS solver for this model

Think of this as creating a blank canvas where we'll paint our optimization problem.

Section 2 - Adding Variables

Now that we have our model, we need to define our variables. In our factory problem, the variables represent the quantities of Product A and Product B that we want to produce.

In JuMP, we use the `@variable` macro to define variables. Here's the general syntax:

```
@variable(model_name, variable_name, [additional_properties])
```

Where:

- `model_name` is the name of your JuMP model
- `variable_name` is what you want to call your variable
- `[additional_properties]` can include bounds or variable types

For example, to create a continuous variable that's greater than or equal to 0:

```
@variable(model_name, variable_name >= 0)
```

Here, `>= 0` sets the lower bound of the variable to zero, while no upper bound is given.

Exercise 2.1 - Create Variables

Now it's your turn! Create two continuous variables equal to or larger than 0 called `productA` and `productB` that represent the number of units produced in our problem for our model `model`.

```
# YOUR CODE BELOW


# Test your answer
@assert @isdefined productA
@assert typeof(productA) == VariableRef
@assert has_upper_bound(productA) == false
@assert has_lower_bound(productA) == true
@assert lower_bound(productA) == 0
@assert @isdefined productB
@assert typeof(productB) == VariableRef
@assert has_upper_bound(productB) == false
@assert has_lower_bound(productB) == true
@assert lower_bound(productB) == 0
println("Variables added to the model successfully!")
```

> **Note**
> Our variables count units produced, but we declared them as continuous variables. That is fine here, as the optimal solution of this problem happens to be whole numbers anyway. The next tutorial shows how to declare integer variables.

Section 3 - Adding Constraints

Constraints are conditions that limit the possible values of the variables in our optimization problem. In JuMP, we use the `@constraint` macro to define these constraints.

The general syntax for adding a constraint is:

```
@constraint(model_name, constraint_name, constraint_expression)
```

Where:

- `model_name` is the name of your JuMP model
- `constraint_name` is a label you give to the constraint (optional, but useful for reference)
- `constraint_expression` is the mathematical expression of the constraint

For example:

```
@constraint(model_name, constraint_name, 4 * variable_name <= 100)
```

This defines a constraint that ensures that the variable `variable_name` can be at most 25. Note that you will have to change `model_name`, `constraint_name` and `variable_name` according to your instance.

Exercise 3.1 - Create Constraints

Create two constraints based on the Cutting and Finishing department hours of the problem description in this tutorial. Call the first constraint `cutting_constraint` and the second constraint `finishing_constraint`.

```
# YOUR CODE BELOW


# Test your answer
@assert is_valid(model, cutting_constraint)
@assert is_valid(model, finishing_constraint)
println("Constraints added to the model successfully!")
println("Note that only the existence of these constraints was checked!")
println("The optimization later will show whether the formulation was
correct.")
```

Section 4 - Defining the Objective Function

The objective function represents the goal of our optimization problem - what we want to maximize or minimize. In JuMP, we use the `@objective` macro to define this function.

The general syntax for defining an objective function is:

```
@objective(model_name, optimization_direction, objective_expression)
```

Where:

- `model_name` is the name of your JuMP model
- `optimization_direction` is either `Max` for maximization or `Min` for minimization

- `objective_expression` is the mathematical expression of the objective function

For example:

```
@objective(model_name, Max, 2*variableA + 3*variableB)
```

This defines an objective function that maximizes something based on the values of `variableA` and `variableB`.

Exercise 4.1 - Create the Objective Function

Create the objective function based on the problem description of this tutorial. The objective is to maximize profit based on the values of `productA` and `productB`.

```
# YOUR CODE BELOW


# Test your answer
@assert objective_sense(model) == MOI.MAX_SENSE "The objective should be
maximized. Did you define it with @objective(model, Max, ...)?"
@assert coefficient(objective_function(model), productA) != 0 "The objective
should include productA."
@assert coefficient(objective_function(model), productB) != 0 "The objective
should include productB."
println("An objective function defined successfully!")
println("The optimization later will show whether the formulation was
correct.")
```

Section 5 - Solving the Model

Now that we have defined our variables, constraints, and objective function, we're ready to solve the optimization model. Here's how to do it step by step:

1. Solve the model:

```
optimize!(model)
```

2. Check the status of the solution:

```
termination_status(model)
```

3. If the solution is optimal, we can retrieve the optimal values of our variables and the objective function. If the time limit is reached, we can check whether a primal solution is available. We do so by checking whether the model has values.

```
begin
    if termination_status(model) == OPTIMAL
        println("Solution is optimal")
```

```
    elseif termination_status(model) == TIME_LIMIT && has_values(model)
        println("Solution is suboptimal due to a time limit,
            but a primal solution is available")
    else
        error("The model was not solved correctly.")
    end
    println("Objective value = ", objective_value(model))
end
```

We can then get the values of the variables as follows:

```
println("Product A quantity: $(value(productA))")
println("Product B quantity: $(value(productB))")
```

Let's break this down:

- `optimize!(model)` tells JuMP to solve our model
- `termination_status(model)` checks if we found an optimal solution
- `objective_value(model)` gives us the maximum profit we can achieve
- `value(productA)` and `value(productB)` tell us how many of each product we should produce

The following code block tests whether the solution is correct or whether you have made a mistake in the formulation of the problem.

> **Note**
> The values might be slightly off due to the nature of floating-point numbers in computers. That's why the tests below use `isapprox` (approximately equal) with a small tolerance `atol` instead of `==`.

```
# Test your answer
@assert termination_status(model) == OPTIMAL "Sorry, something didn't work out
as the model status is $(termination_status(model))"
println("Solution: Product A = ", value(productA), ", Product B = ",
value(productB))
@assert isapprox(value(productA), 12; atol=1e-4) "Although you have a solution,
productA should be 12 not $(value(productA))"
@assert isapprox(value(productB), 4; atol=1e-4) "Although you have a solution,
productB should be 4 not $(value(productB))"
@assert isapprox(objective_value(model), 1800; atol=1e-4) "The quantities are
correct, but the objective value should be 1800 not $(objective_value(model))"
println("You have solved the model correctly!")
```

If everything worked, you found the optimal production plan: 12 units of Product A and 4 units of Product B, for a maximum profit of $100 \cdot 12 + 150 \cdot 4 = 1800$. Note that both departments are fully used: Cutting takes $2 \cdot 12 + 4 \cdot 4 = 40$ of the 40 available hours and Finishing takes $4 \cdot 12 + 3 \cdot 4 = 60$ of the 60 available hours.

> **Question: Why don't we produce only Product B?**
> Product B earns 150 per unit and Product A only 100, so producing only Product B sounds tempting. But Product B also uses up the Cutting hours twice as fast as Product A. With only Product B, the Cutting department limits us to $40/4 = 10$ units, for a profit of $10 \cdot 150 = 1500$. Mixing both products uses the hours of both departments fully and earns more: 1800.

Conclusion

Excellent! You've completed the tutorial on mathematical optimization with JuMP and HiGHS. You've learned how to define optimization models, add variables and constraints, define an objective function, and solve the model. Continue to the next file to learn more.

Solutions

You will likely find solutions to most exercises online. However, I strongly encourage you to work on these exercises independently without searching explicitly for the exact answers to the exercises. Understanding someone else's solution is very different from developing your own. Use the lecture notes and try to solve the exercises on your own. This approach will significantly enhance your learning and problem-solving skills.

Remember, the goal is not just to complete the exercises, but to understand the concepts and improve your programming abilities. If you encounter difficulties, review the lecture materials, experiment with different approaches, and don't hesitate to ask for clarification during class discussions.

Bibliography